

Huffman编码树： 算法

编码

❖ 通讯 / 编码 / 译码

❖ 二进制编码

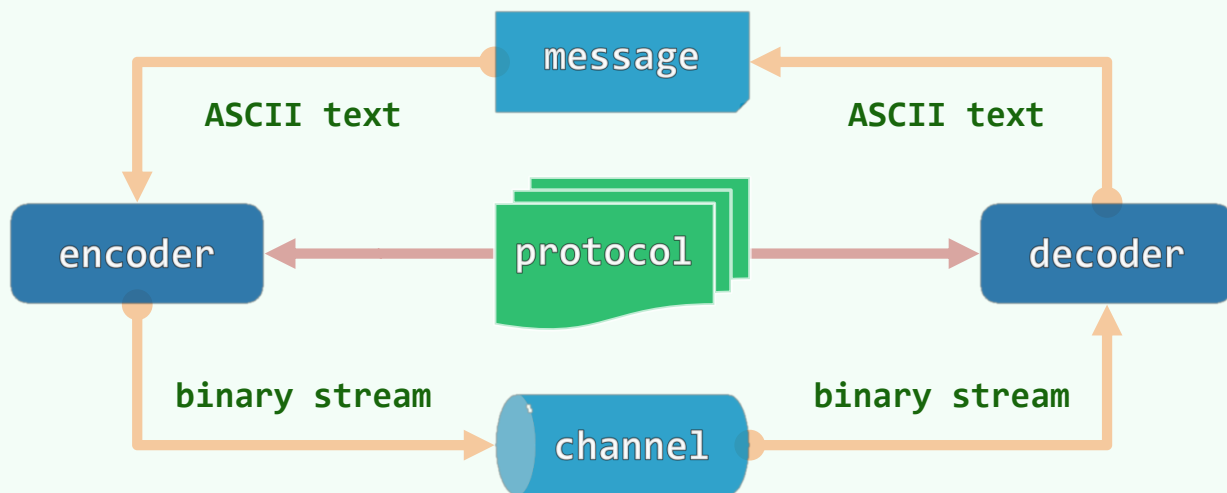
- 组成数据文件的字符来自字符集 Σ
- 字符被赋予互异的二进制串

❖ 文件的大小取决于

- 字符的数量 $\times$ 各字符编码的长短

❖ 通讯带宽有限时

- 如何对各字符编码，使文件最小？



1 010 011 00 ~ MAIN

M	A	I	N
1	010	011	00

PFC编码

❖ 将 Σ 中的字符组织成一棵二叉树，以0/1表示左/右孩子

各字符 x 分别存放于对应的叶子 $v(x)$ 中

❖ 字符 x 的编码串 $rps(v(x)) = rps(x)$

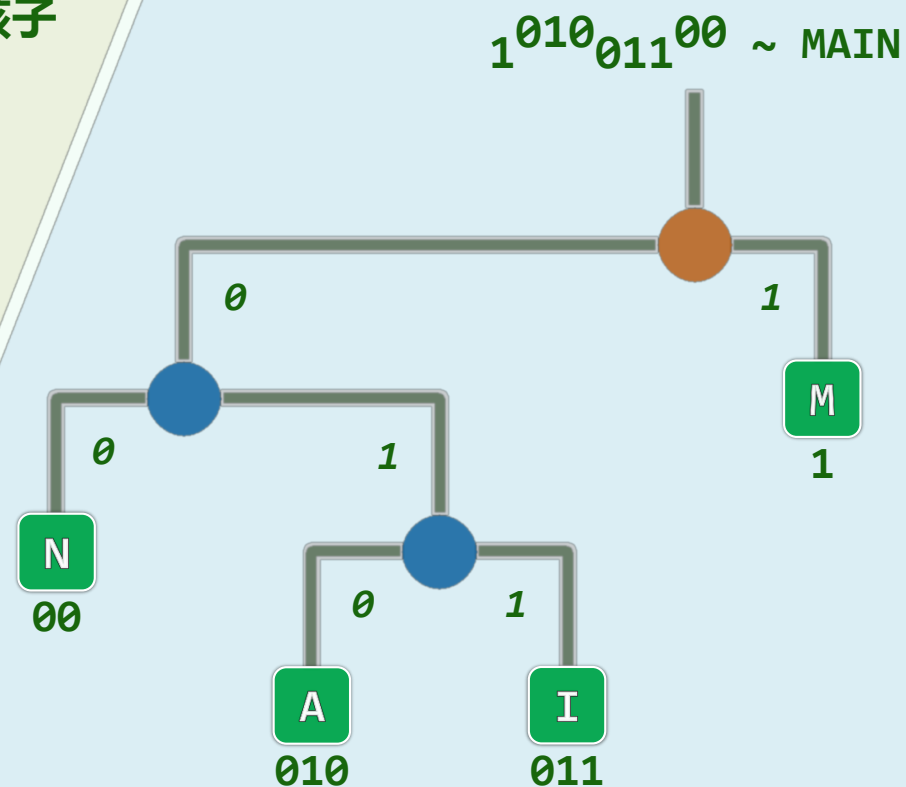
由根到 $v(x)$ 的通路 (root path) 确定

❖ 字符编码**不必**等长，而且...

❖ 不同字符的编码**互不为前缀**，故不致歧义

(Prefix-Free Code)

❖ 缺点：你能发现吗？



编码长度 vs. 叶节点平均深度

❖ 平均编码长度

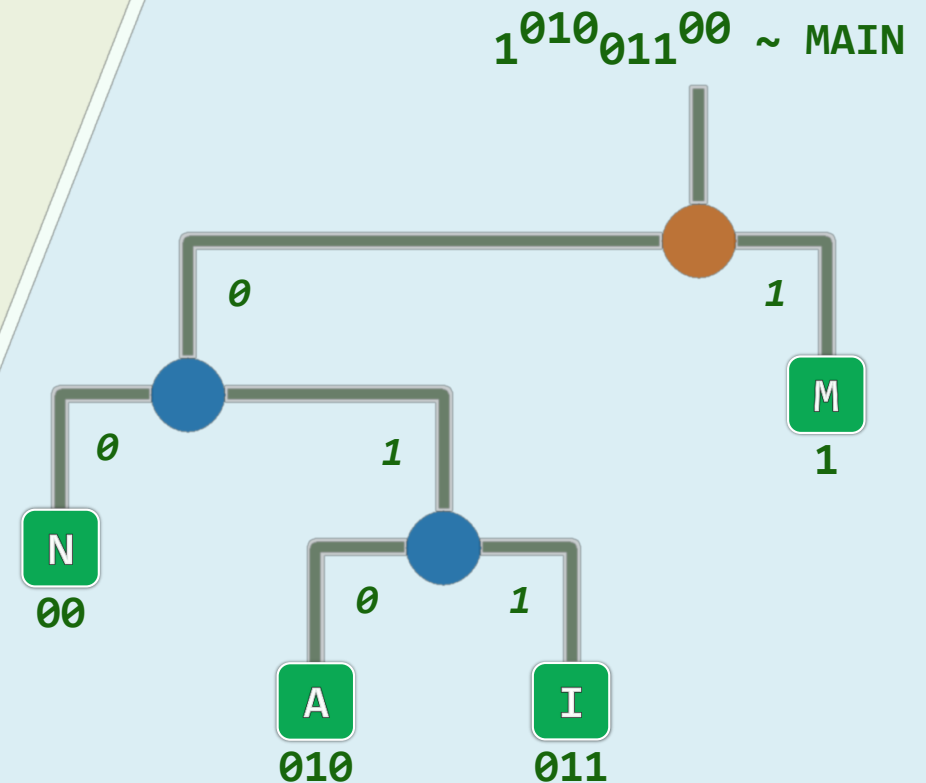
$$\text{ald}(T) = \sum_{x \in \Sigma} \text{depth}(v(x)) / |\Sigma|$$

❖ 对于特定的 Σ

$\text{ald}()$ 最小者即为最优编码树 T_{opt}

❖ 最优编码树必然存在，但不见得唯一

它们具有哪些特征？



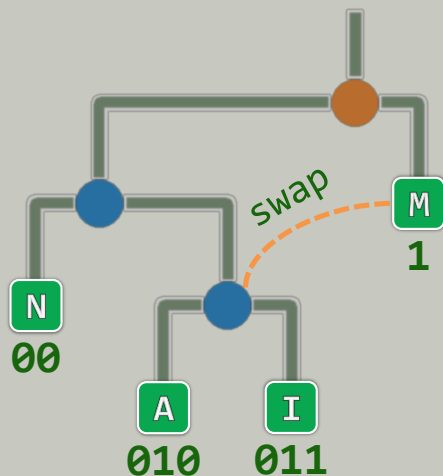
最优编码树

$\forall v \in T_{\text{opt}}, \deg(v) = 0$ **only if** $\text{depth}(v) \geq \text{depth}(T_{\text{opt}}) - 1$

亦即，叶子只能出现在**倒数两层**以内——否则，通过节点**交换**即可...

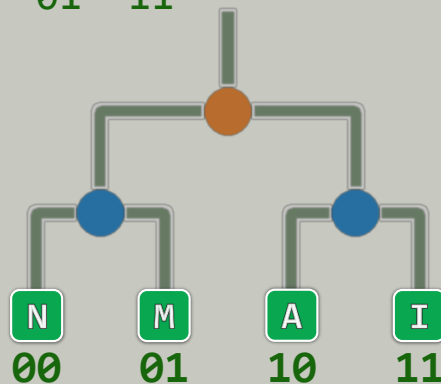
$$\text{ald}(T) * 4 = 2 + 3 + 3 + 1 = 9$$

"101001100" = "MAIN"



$$\text{ald}(T) * 4 = 2 + 2 + 2 + 2 = 8$$

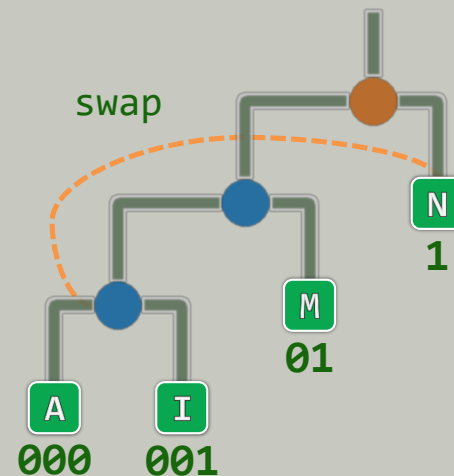
"01101100" = "MAIN"



特别地，**真**完全树即是**最优**编码树

$$\text{ald}(T) * 4 = 2 + 3 + 3 + 1 = 9$$

"01000011" = "MAIN"



字符频率

❖ 实际上，字符的出现**概率或频度**不尽相同

甚至，往往相差极大...

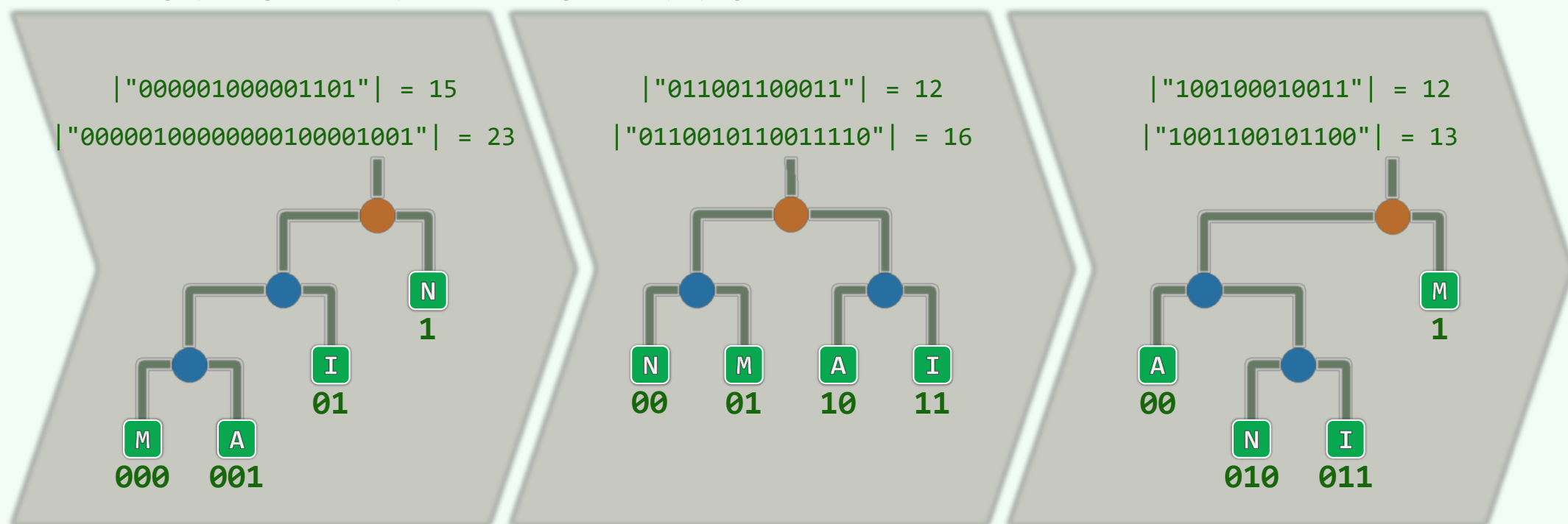


❖ 已知各字符的**期望频率**，如何构造最优编码树？

带权编码长度 vs. 叶节点平均带权深度

❖ 文件长度 $\propto$ 平均带权深度 $wald(T) = \sum_x rps(x) \times w(x)$

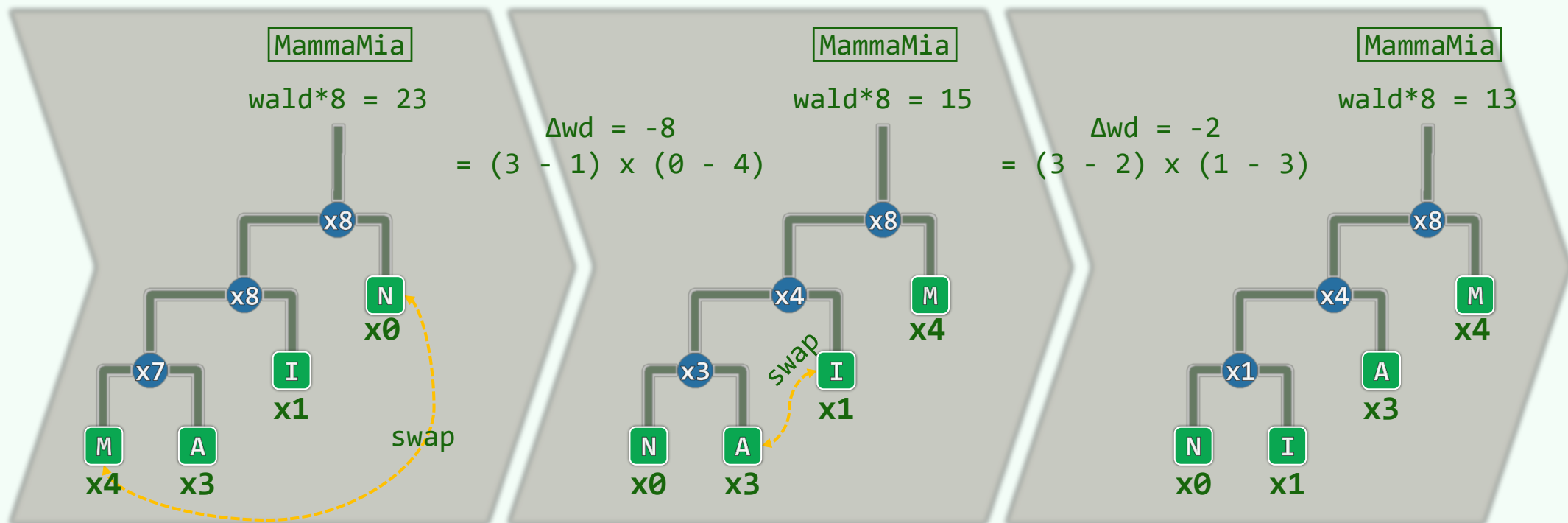
❖ 此时，完全树**未必**就是最优编码树——比如，考查"mamani"和"mammamia"...



最优带权编码树

❖ 同样，频率高/低的（超）字符，应尽可能放在高/低处

❖ 故此，通过适当交换，同样可以缩短 $wald(T)$



Huffman算法

// 贪婪策略：频率低的字符优先引入，位置亦更低

为每个字符创建一棵单节点的树，组成森林F

按照出现频率，对所有树排序

while (F中的树不止一棵)

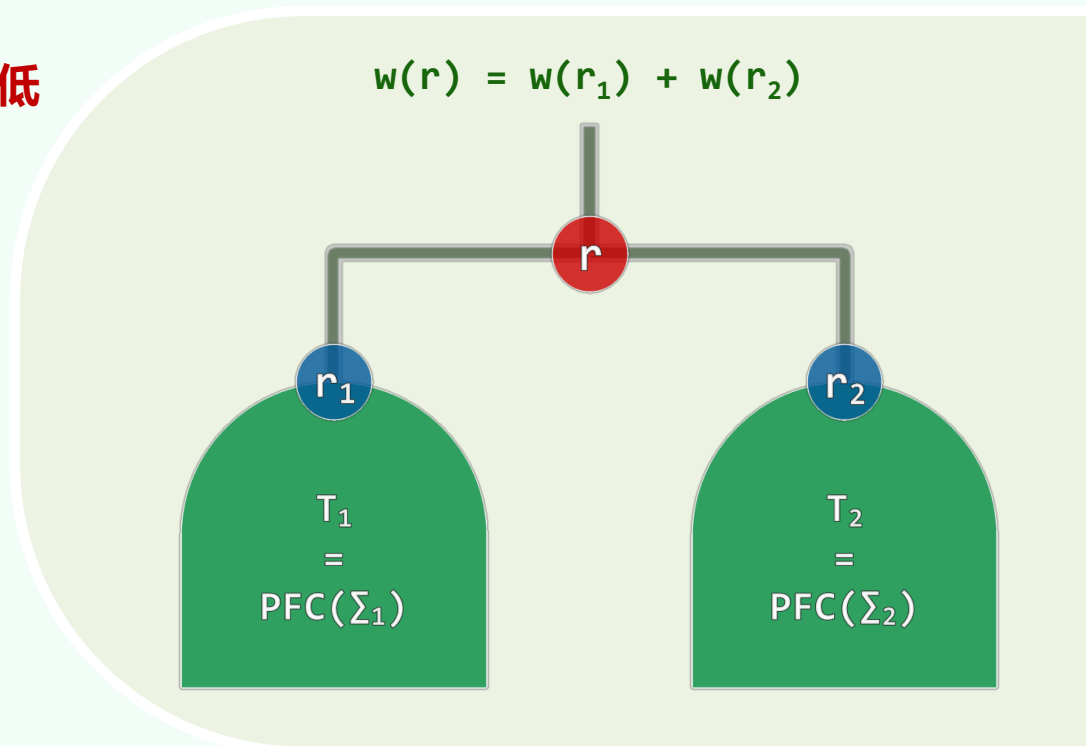
取出频率最小的两棵树： T_1 和 T_2

将它们合并成一棵新树T，并令：

$$lc(T) = T_1 \text{ 且 } rc(T) = T_2$$

$$w(\text{root}(T)) = w(\text{root}(T_1)) + w(\text{root}(T_2))$$

// 尽管贪心策略未必总能得到最优解，但非常幸运，如上算法的确能够得到最优编码树之一

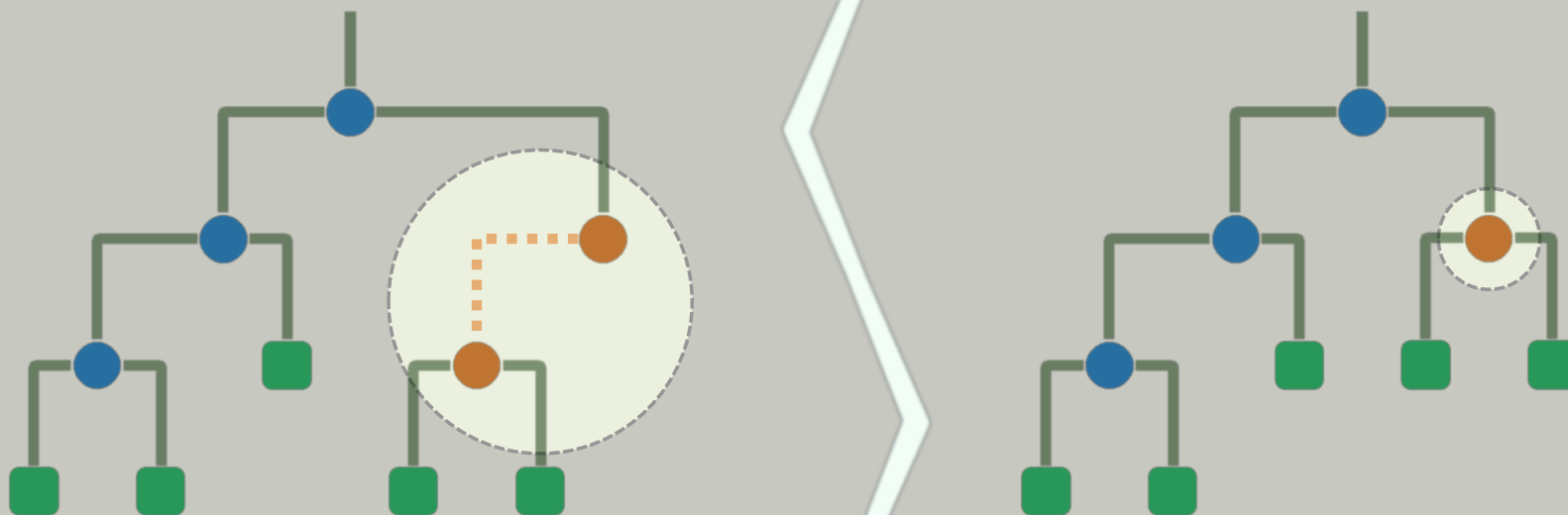


双子性

❖ 最优编码树有何特征？

❖ 首先，每一内部节点都有**两个**孩子——节点度数均为偶数（0或2），即**真**二叉树

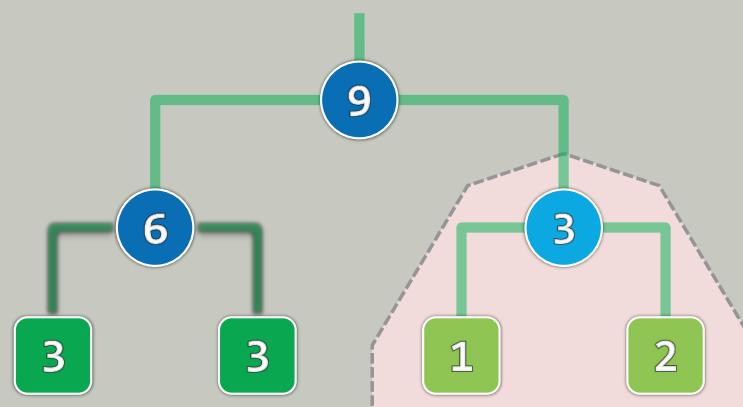
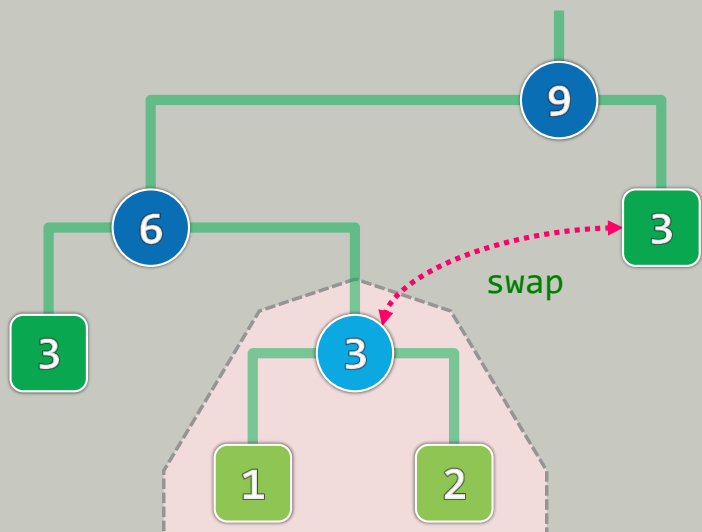
❖ 否则，将1度节点**替换**为其唯一的孩子，则新树的wald将**更小**



不唯一性

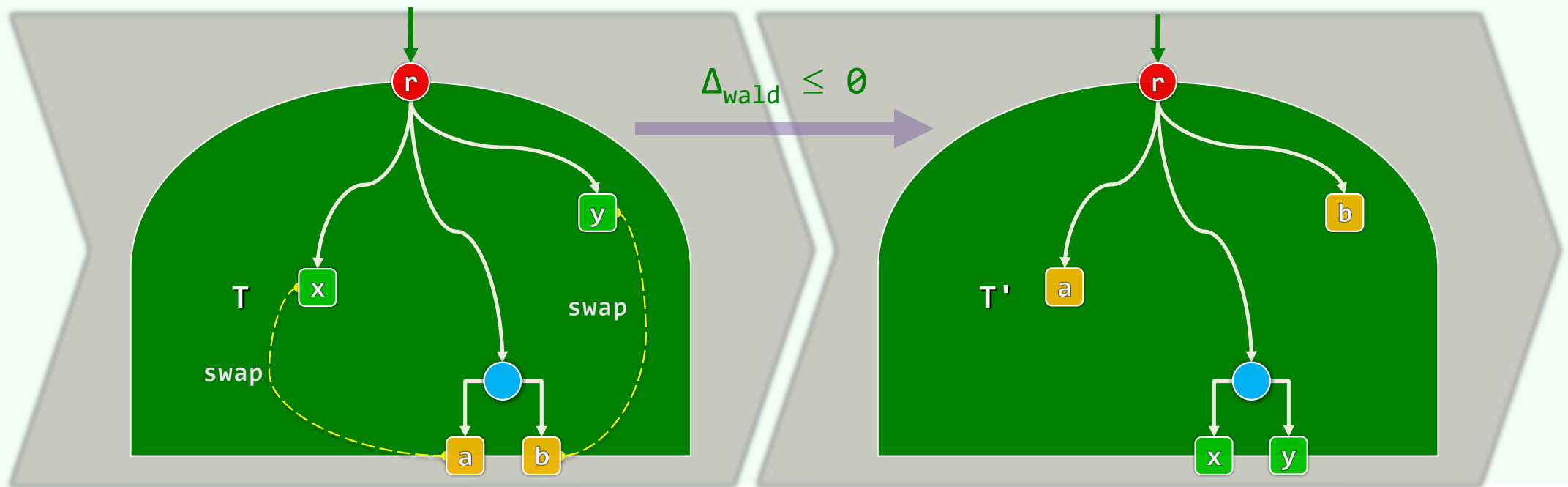
- ❖ 对任一内部节点而言
左、右子树**互换**之后wald不变
- ❖ 上述算法中，**兄弟子树的次序系随机**选取
故有可能...

- ❖ 为消除这种歧义，可以（比如）
明确要求**左子树的频率更低**
- ❖ 不过，倘若
它们（甚至更多节点）的频率恰好相等...



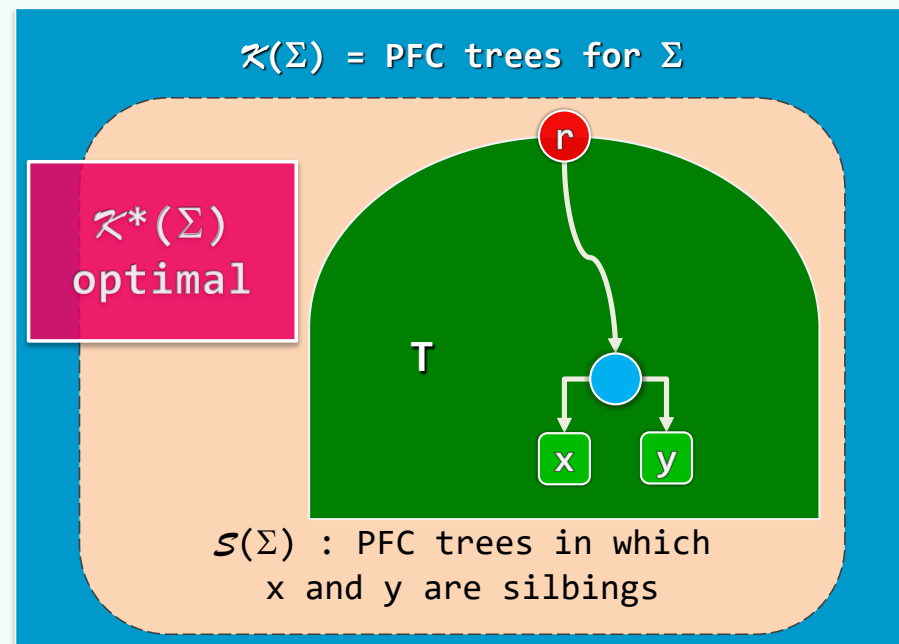
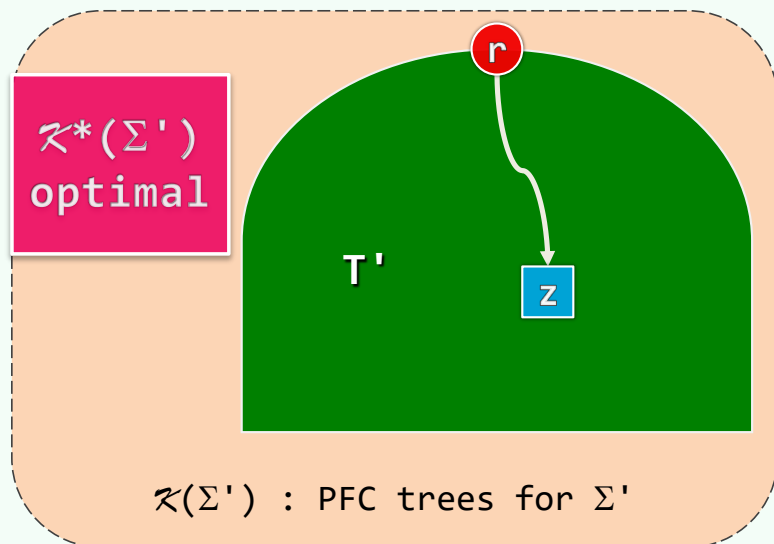
层次性

- ❖ 出现**频率最低**的字符 x 和 y ，必在**某棵**最优编码树中处于**最底层**，且互为**兄弟**
 - ❖ 否则，**任取**一棵最优编码树，并在其最底层**任取**一对兄弟 a 和 b
- 于是， a 和 x 、 b 和 y 交换之后， $wald$ 绝不会增加



数学归纳

- ❖ 对 $|\Sigma|$ 做归纳可证：Huffman 算法所生成的，必是一棵**最优**编码树！ $|\Sigma| = 2$ 时显然
- ❖ 设算法在 $|\Sigma| < n$ 时均正确。现设 $|\Sigma| = n$ ，取 Σ 中频率最低的 x 、 y （不妨就设二者互为兄弟）
- ❖ 令： $\Sigma' = (\Sigma \setminus \{x, y\}) \cup \{z\}$ ， $w(z) = w(x) + w(y)$

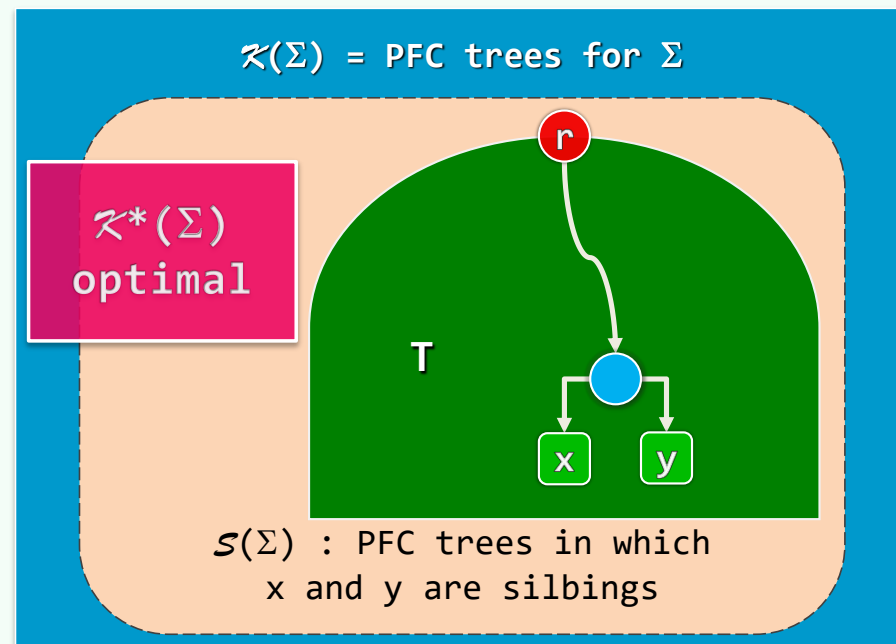
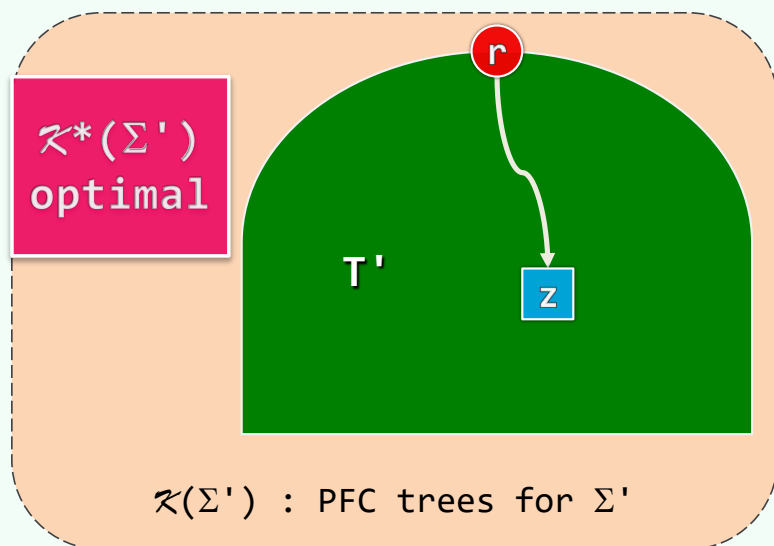


定差

❖ 对于 Σ' 的任一编码树 T' ，只要为 z 添加孩子 x 和 y ，即可得到 Σ 的一棵编码树 T ，且

$$wd(T) - wd(T') = w(x) + w(y) = w(z)$$

❖ 可见，如此对应的 T 和 T' ， wd 之差与 T 的具体形态无关

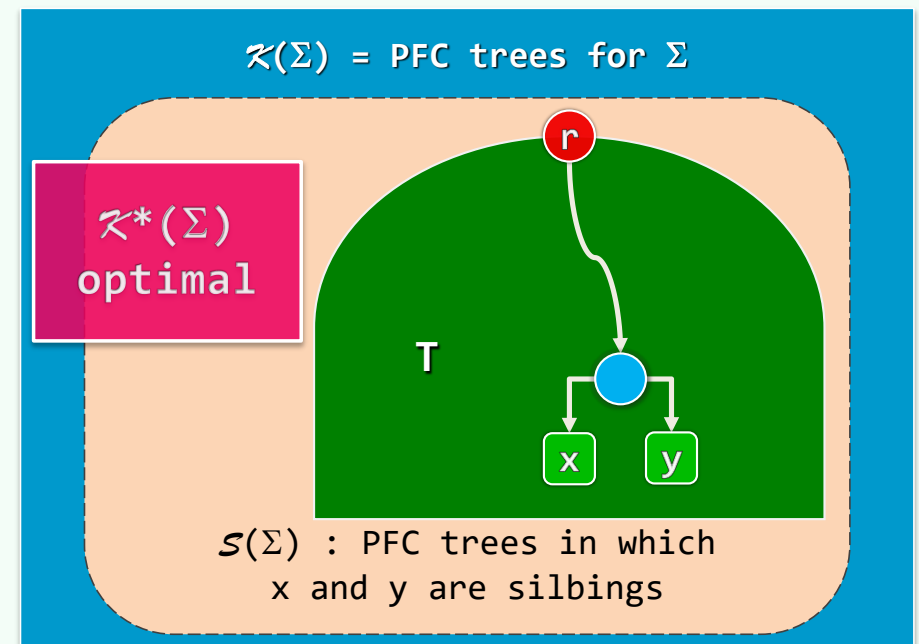
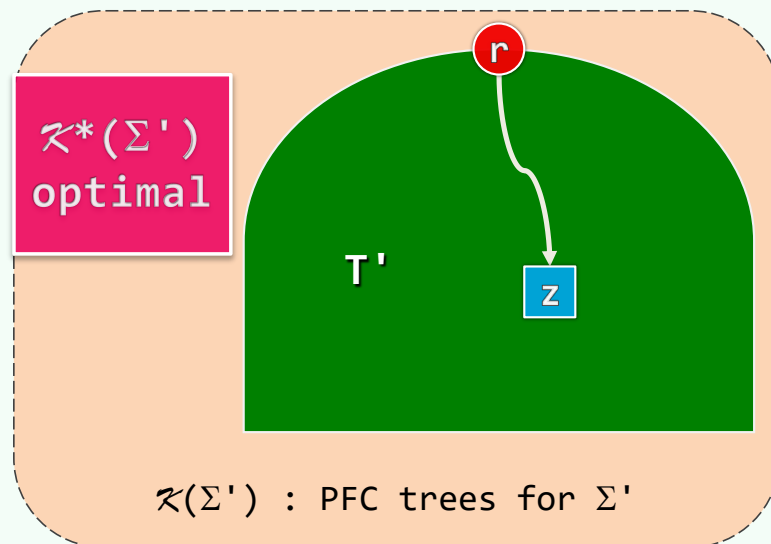


从最优，到最优

❖ 因此，只要 T' 是 Σ' 的最优编码树，则 T 也必是 Σ 的最优编码树（之一）

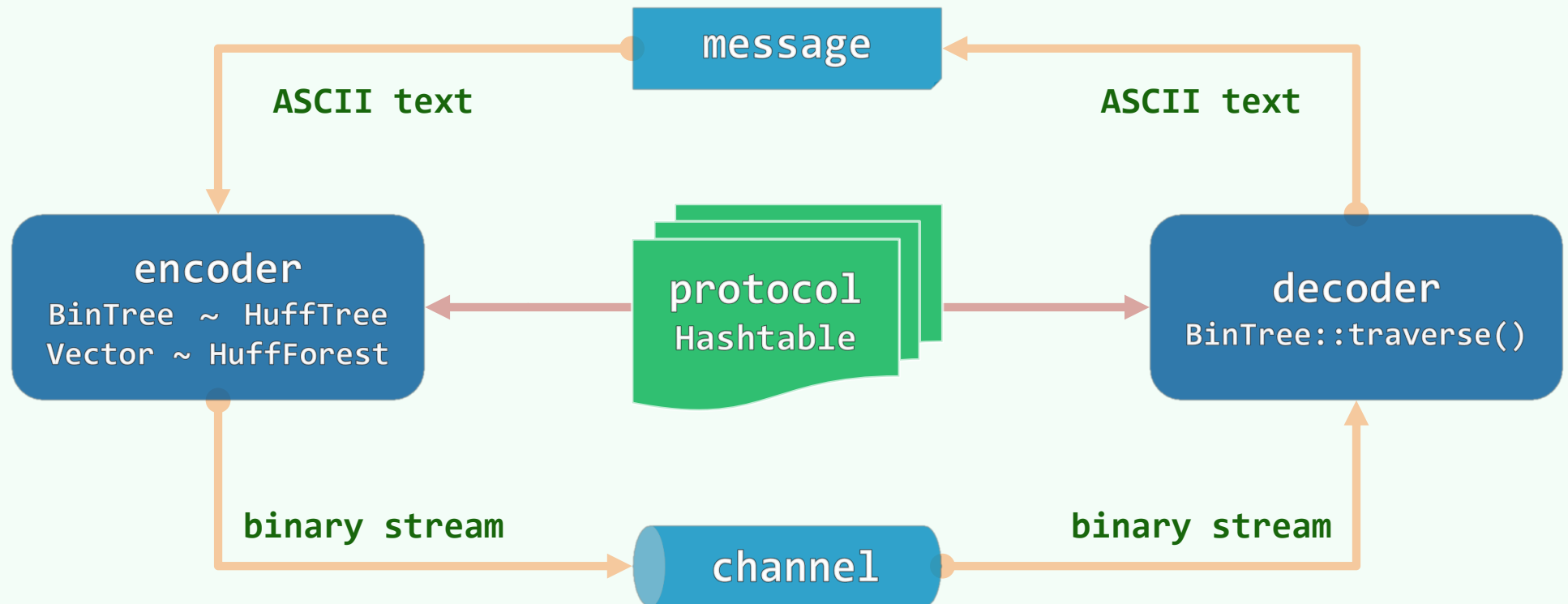
❖ 实际上，Huffman算法的过程，与上述归纳过程完全一致

——每一步迭代都可视作，从某棵 T 转入对应的 T'



Huffman编码树： 算法实现

数据结构与算法



Huffman (超) 字符

```
#define N_CHAR (0x80 - 0x20) //仅以可打印字符为例
```

```
struct HuffChar { //Huffman (超) 字符
```

```
    char ch; int weight; //字符、频率
```

```
    HuffChar ( char c = '^', int w = 0 ) : ch ( c ), weight ( w ) {};
```

```
    bool operator< ( HuffChar const& hc ) { return weight > hc.weight; } //比较器
```

```
    bool operator== ( HuffChar const& hc ) { return weight == hc.weight; } //判等器
```

```
};
```

Huffman树与森林

❖ Huffman (子) 树

```
using HuffTree = BinTree< HuffChar >;
```

❖ Huffman森林

```
using HuffForest = List< HuffTree* >;
```

❖ 待日后掌握了更多数据结构之后，可改用更为高效的方式，比如：

```
using HuffForest = PQ_List< HuffTree* >; //基于列表的优先级队列
```

```
using HuffForest = PQ_ComplHeap< HuffTree* >; //完全二叉堆
```

```
using HuffForest = PQ_LeftHeap< HuffTree* >; //左式堆
```

❖ 得益于已定义的统一接口，支撑Huffman算法的这些底层数据结构可直接彼此替换

构造编码树

```
HuffTree* generateTree( HuffForest * forest ) { //Huffman编码算法
```

```
while ( 1 < forest->size() ) { //反复迭代, 直至森林中仅含一棵树
```

```
    HuffTree *T1 = minHChar( forest ), *T2 = minHChar( forest );
```

```
    HuffTree *S = new HuffTree(); //创建新树, 准备合并T1和T2
```

```
    S->insert( HuffChar( '^', //根节点权重, 取作T1与T2之和
```

```
        T1->root()->data.weight + T2->root()->data.weight ) );
```

```
    S->attach( T1, S->root() ); S->attach( S->root(), T2 );
```

```
    forest->insertAsLast( S ); //T1与T2合并后, 重新插回森林
```

```
} //assert: 循环结束时, 森林中唯一的那棵树即Huffman编码树
```

```
return forest->first()->data; //故直接返回之
```

```
}
```

查找最小超字符

❖ Huffman编码的整体效率，直接决定于minHChar()的效率

```
HuffTree* minHChar( HuffForest * forest ) { //此版本仅达到 $O(n)$ ，故整体为 $O(n^2)$ 
```

```
    ListNodePosi( HuffTree* ) p = forest->first(); //从首节点出发
```

```
    ListNodePosi( HuffTree* ) minChar = p; //记录最小树的位置及其
```

```
    int minWeight = p->data->root()->data.weight; //对应的权重
```

```
    while ( forest->valid( p = p->succ ) ) //遍历所有节点
```

```
        if( minWeight > p->data->root()->data.weight ) { //如必要，则
```

```
            minWeight = p->data->root()->data.weight; minChar = p; //更新记录
```

```
        }
```

```
    return forest->remove( minChar ); //从森林中摘除该树，并返回
```

```
}
```

构造编码表

```
#include "Hashtable.h" //用HashTable (第09章) 实现
using HuffTable = Hashtable< char, char* >; //Huffman编码表

static void generateCT //通过遍历获取各字符的编码
( Bitmap* code, int length, HuffTable* table, BinNodePosi( HuffChar ) v ) {
    if ( IsLeaf( * v ) ) //若是叶节点 (还有多种方法可以判断)
        { table->put( v->data.ch, code->bits2string( length ) ); return; }
    if ( HasLChild( * v ) ) //Left = 0, 深入遍历
        { code->clear( length ); generateCT( code, length + 1, table, v->lc ); }
    if ( HasRChild( * v ) ) //Right = 1
        { code->set( length ); generateCT( code, length + 1, table, v->rc ); }
} //总体O(n)
```

Huffman编码树：改进

向量 + 列表 + 优先级队列

- ❖ 方案1, $O(n^2)$:
 - 初始化时, 通过排序得到一个**非升序向量** $//O(n\log n)$
 - 每次 (从**后端**) 取出频率最低的两个节点 $//O(1)$
 - 将合并得到的新树插入向量, 并保持有序 $//O(n)$
- ❖ 方案2, $O(n^2)$:
 - 初始化时, 通过排序得到一个**非降序列表** $//O(n\log n)$
 - 每次 (从**前端**) 取出频率最低的两个节点 $//O(1)$
 - 将合并得到的新树插入列表, 并保持有序 $//O(n)$
- ❖ 方案3, $O(n\log n)$:
 - 初始化时, 将所有树组织为一个**优先级队列** (第12章) $//O(n)$
 - 取出频率最低的两个节点, 合并得到的新树插入队列 $//O(\log n) + O(\log n)$

预排序 x (栈 + 队列)

- ❖ 方案4: - 所有字符按频率排序, 构成一个栈 $//O(n\log n)$
- 维护另一个有序队列... $//O(n)$

